

11/7/2025

classmate

Date _____

Page _____

- Practical-1.

① WAP to declare a class student having data members as roll, name. Accept and display data for one object.

Ans:

```
#include <iostream>
using namespace std;
class Student;
{
    private:
        char name[10];
        int roll;
    public:
        void accept ()
        {
            cout << "Name";
            cin >> name;
            cout << "Roll no.:";
            cin >> roll;
        }
        void disp ()
        {
            cout << roll;
            cout << " , " << name;
        }
};

int main ()
{
    Student s1;
    s1.accept();
    s1.display();
    return 0;
}
```

- (b) WAP to declare a class book having data members id, name, price. Accept data for 2 books and display name of book having greater price.

Ans:

```
#include <iostream>
using namespace std;
class Book
{
private:
    int id;
    char name[20];
public:
    int price;
    void accept()
    {
        cout << "Name: ";
        cin >> name;
        cout << "Id: ";
        cin >> id;
        cout << "Price: ";
        cin >> price;
    }
    void disp()
    {
        cout << name;
        cout << ", " << id;
        cout << ", " << price << "\n";
    }
};
```



```
int main()
{
    Book B1, B2;
    B1.accept();
    B2.accept();
    if (B1.price > B2.price)
    {
        cout << "Book having greater price is B1.";
        cout << "\n";
        B1.disp();
    } else if (B1.price < B2.price) {
        cout << "Book having greater price is B2.";
        cout << "\n";
        B2.disp();
    } if (B1.price == B2.price) {
        cout << "Both books prices are same.";
        cout << "\n";
        B1.disp();
        cout << "\n";
        B2.disp();
    }
    return 0;
}
```

© WAP to declare class Time. Accept time in HH:MM:SS and display total time in seconds.

Ans:

```
#include <iostream>
using namespace std;
class Time
{
    private:
        int min, sec, hr;
    public:
        int i;
        void accept()
        {
            cout << "Hours:";
            cin << hr;
            cout << "Minutes:";
            cin << min;
            cout << "Seconds:";
            cin << sec;
            i = (hr * 3600) + (min * 60) + sec;
        }
        void disp()
        {
            cout << hr << ":" << min << ":" << sec;
        }
};

int main() {
    Time T1;
    T1.accept();
    cout << "Time converted in seconds is ";
    return 0;
}
```

7/8

18/7/25

classmate

Date _____

Page _____

- Practical-2

Q. Write a C++ program to demonstrate the use of array of objects.

Q. WAP to declare a class 'city' having data members as name and population. Accept this data for 5 cities and display name of city having highest population.

Ans:

```
#include <iostream>
using namespace std;
class city
{
    private:
        char name[20];
    public:
        int population;
        void accept()
        {
            cout << "Enter city name: ";
            cin >> name;
            cout << "City's population: ";
            cin >> population;
        }
        void disp() {
            cout << "City with highest population: " << name;
        }
};

int main() {
    city c[5], max;
    int i;
    for (i = 0; i < 5; i++) {
        c[i].accept();
    }
```

```

max = c[0];
for (i = 1; i < 5; i++) {
    if (max.population < c[i].population) {
        max = c[i];
    }
}
max.display();
return 0;
}

```

- (b) WAP to declare a class 'Account' having data members as Account no. and balance. Accept this data for 10 accounts and give interest of 10% where balance is equal or greater than 5000 and display them.

Ans:

```

#include <iostream>
using namespace std;
class Account {

```

private:

int accountno;

public:

int balance;

void accept();

{

cout << "Enter Account no: ";

cin >> accountno;

cout << "Enter account balance: ";

cin >> balance;

}


```
void disp()
```

```
{
```

```
    cout << "Balance after 1 year of interest: " << balance  
    << "\n";
```

```
}
```

```
}
```

```
int main()
```

```
{
```

```
    Account a[10];
```

```
    int i;
```

```
    for(i=0; i<10; i++){
```

```
        a[i].accept();
```

```
}
```

```
    for(i=0; i<10; i++){
```

```
        if(a[i].balance >= 5000){
```

```
            a[i].balance += (a[i].balance * 10 * 1) / 100;
```

```
        }
```

```
    }
```

```
    for(i=0; i<10; i++){
```

```
        a[i].disp();
```

```
    }
```

```
    return 0;
```

```
}
```

- © WAP to declare a class 'Staff' having data members as name and post. Accept this data for 5 staff and display names of staff who are "HOD".

Ans:

```
#include <iostream>
using namespace std;
class Staff
{
    private:
    string name;
    public:
    string post;
    void accept()
    {
        cout << "Enter employee name: ";
        cin >> name;
        cout << "Employee post: ";
        cin >> post;
    }
    void disp() {
        cout << "Employee with HOD post: " << name << "\n";
    }
};

int main() {
    Staff c[5];
    int i;
    for (i = 0; i < 5; i++) {
        c[i].accept();
    }
    for
```



```
for (i=0; i<5; i++) {  
    if (c[i].post == "HOD") {  
        c[i].disp();  
    }  
}  
return 0;  
}
```

Pr
7/8

25/7/25

classmate

Date _____
Page _____

- Practical - 3

Q. Pointer to object, The 'this' pointer, nested class.

Q. Write a program to declare a class 'book' containing data members as book-title, author_name and price. Accept and display the information for one object using a pointer to that object.

Ans:

```
#include <iostream>
using namespace std;
{
    private:
        char book_title[20];
        char Author_name[20];
        int price;
        void accept() {
            cout << "Book name: ";
            cin >> book_title;
            cout << "Author name: ";
            cin >> author_name;
            cout << "Book price: ";
            cin >> price;
        }
        void disp() {
            cout << "Book name is " << book_title << endl;
            cout << "Author name is " << author_name << endl;
            cout << "Book price is " << price;
        }
    }
};
```



```

int main() {
    Book B1;
    Book *p;
    p = &B1;
    p → accept();
    p → disp();
    return 0;
}

```

- (b) WAP to declare a class 'student' having data members roll_no and percentage. Using 'this' pointer invoke member functions to accept and display this data for one object of the class.

Ans:

```

#include <iostream>
using namespace std;
class Student
{
private:
    int roll;
    float per;
public:
    void accept() {
        cout << "Enter roll no: ";
        cin >> roll;
        cout << "Enter percentage: ";
        cin >> per;
    }
    void disp() {
        cout << "Roll no: " << roll
        this → accept();
        cout << "Roll no: " << roll;
        cout << "Percentage: " << per;
    }
}

```

```

    }
};
int main () {
    Student s1;
    s1.disp();
    return 0;
}

```

③ Write a program to demonstrate the use of nested class.

Ans:

```

#include <iostream>
using namespace std;
class Student {
    private:
        int roll;
        char name(20);
    public:
        void accept() {
            cout << "Roll no: ";
            cin >> roll;
            cout << "Name: ";
            cin >> name;
        }
        void disp() {
            cout << "Roll no: " << roll;
            cout << " Name: " << name;
        }
        class Marks {
            private:
                int m1, m2;
            public:
                void get(m1, m2)
                m1 = 10;

```



```
m2 = 16;
```

```
void avg() {
```

```
int q = (m1 + m2) / 2;
```

```
cout << "Average: " << q
```

```
}
```

```
};
```

```
};
```

```
int main() {
```

```
Student s1;
```

```
Student::Marks ms;
```

```
s1.accept();
```

```
ms.get();
```

```
s1.display();
```

```
ms ms.avg();
```

```
return 0;
```

```
}
```

PL
11/8

- Practical - 4.

- ① WAP to swap two numbers from same class using object as function argument. Write swap function as member function.

Ans:

```
#include <iostream>
using namespace std;
class Numbers {
private:
    int a, b;
public:
    void accept() {
        cout << "Enter First no: ";
        cin >> a;
        cout << "Enter Second no: ";
        cin >> b;
    }
    void disp() {
        cout << "a = " << a << "b = " << b << endl;
    }
    void swap(Numbers n) {
        int c = n.a;
        int n.a = n.b;
        n.b = c;
    }
};

int main() {
    Numbers N1;
    N1.accept();
    cout << "Before swapping: ";
    N1.disp();
    N1.swap(N1);
    cout << "After swapping: ";
    N1.disp();
    return 0;
}
```


- ② WAP to swap two numbers from ~~same~~ different class using concept of friend function.

Ans:

```
#include <iostream>
using namespace std;
class B;
class A {
    int i;
public:
    void accept () {
        cout << "Enter first no: ";
        cin >> i; }
    void disp () {
        cout << "First no: " << i << endl; }
    friend void swap (class A, class B);
};

class B {
    int j;
public:
    void accept () {
        cout << "Enter second no: ";
        cin >> j; }
    void disp () {
        cout << "Second no: " << j << endl; }
    friend void swap (class A, class B);
};

void swap (class A m, class B n) {
    int p = m.i;
    m.i = n.j;
    n.j = p;
    cout << "Swapping in Function: ";
```

```

cout << "First no: " << m.i << endl;
cout << " Second no: " << n.j << endl; }
int main() {
    class A A1;
    class B B1;
    A1.accept();
    B1.accept();
    A1.disp();
    B1.disp();
    swap(A1, B1);
    cout << "After Swapping: ";
    A1.disp();
    B1.disp();
    return 0;
}

```

② NAP to swap two numbers from same class using concept of friend function.

Ans:

```

#include <iostream>
using namespace std;
class Swap {
    int temp, a, b;
    public:
    swap(int a, int b) {
        this → a = a;
        this → b = b;
        friend void swap(Swap &);
    };
    void swap(Swap &S1) {
        cout << "Before Swapping: " << S1.a << ", " << S1.b;

```



```

s1.temp = s1.a;
s1.a = s1.b;
s1.b = s1.temp;
cout << "After Swapping : " << s1.a << " " << s1.b;
}

```

```

int main() {

```

```

    swap(s(4,6);

```

```

    swap(s);

```

```

    return 0;

```

```

}

```

- ④ WAP to create two classes Result1 and Result2 which stores the marks of the students. Read the value of a marks for both the class objects and compute the average of two results

Ans:

```

#include <iostream>

```

```

using namespace std;

```

```

class Result1 {

```

```

    public:

```

```

    int marks;

```

```

    void accept() {

```

```

        cout << "Enter Marks: ";

```

```

        cin >> marks;

```

```

    }

```

```

};

```

```

class Result2 {

```

```

    public:

```

```

    int marks;

```

```

    void accept() {

```

```

        cout << "Enter Marks: ";

```

```

        cin >> marks;

```

```

    }
};
int main() {
    Result r1;
    Result r2;
    r1.accept();
    r2.accept();
    float avg = (r1.marks + r2.marks) / 2;
    cout << "Average marks: " << avg << endl;
    return 0;
}

```

- ⑤ WAP to find the greatest number among two numbers from two different classes using friend function.

Ans:

```

#include <iostream>
using namespace std;
class Number 2;
class Number 1 {
    private:
        int num1;
    public:
        void accept() {
            cout << "Enter first number: ";
            cin >> num1;
        }
        friend void findGreatest (Number 1, Number 2);
};
class Number 2 {
    private:
        int num2;
    public:

```



```
void accept(){
```

```
    cout << "Enter second number: ";
```

```
    cin >> num2; } findGreatest
```

```
friend void friendGreatest (Number1, Number2);
```

```
};
```

```
void findGreatest (Number1 n1, Number2 n2) {
```

```
    if (n1.num1 > n2.num2) {
```

```
        cout << "Greatest number is: " << n1.num1 << endl; }
```

```
    else if (n2.num2 > n1.num1) {
```

```
        cout << "Greatest number is: " << n2.num2 << endl; }
```

```
    else { cout << "Both numbers are equal." << endl; }
```

```
int }
```

```
int main() {
```

```
    Number Number1 obj1;
```

```
    Number2 obj2;
```

```
    obj1 obj1.accept();
```

```
    obj2.accept();
```

```
    findGreatest (obj1, obj2);
```

```
    return 0;
```

```
}
```

11/8

- Practical - 5

- Q. Write a C++ program to implement types of constructors.
- ① Write a program to find the sum of numbers betⁿ 1 to n using a constructor where the value of n will be passed to the constructor.

Ans:

#include <iostream>

using namespace std;

class Number {

int n;

public:

Number() {

n = 5;

}

Number(int num) {

~~num = n;~~ num = n;

}

Number(Number n1) {

n = n1.n;

}

void get() {

cout << "Value = " << n << endl;

for (i = 1; i <= n; i++) {

s += i;

}

cout << "Value = " << s << endl;

}

};

int main() {

Number N2();

Number N3(6);

Number N4(N2);

~Number() {

cout << "Destroyed. " << endl;

}

};


```
N2.get(); int main() {  
N3.get();   Number N2();  
N4.get();   Number N3(6);  
             Number N4(N2);  
             N2.get();  
             N3.get();  
             N4.get();  
             return 0;  
}
```

Output:-

Value = 15

Value = 21

Value = 15

Destroyed.

Destroyed.

Destroyed.

- ② Write a program to declare a class "student" having data members as name and percentage. Write a constructor to initialize these data members. Accept and display data for one student.

Ans:

- Default Constructor:

```
#include <iostream>
```

```
using namespace std;
```

```
class Student{
```

```
    char name[20];
```

```
    float percentage;
```

```
public:
```

```
    Student(){
```

```
        name = "Omkar";
```

```
        percentage = 90;}
```

```
    void disp(){
```

```
        cout << "Name: " << name;
```

```
        cout << "Percentage: " << percentage;}
```

```
};
```

```
int main(){
```

```
    Student s1();
```

```
    s1.disp();
```

```
    return 0;
```

```
}
```

Output:

```
~Student(){
```

```
    cout << "Destroyed.";
```

```
};
```

```
int main(){
```

```
    Student s1();
```

```
    s1.disp();
```

```
    return 0;
```

```
}
```

Output: Name: Omkar

Percentage: 90

Destroyed.

- Parameterized ~~Destructor~~ Constructor:-

#include <iostream>

using namespace std;

class Student {

char name[20];

float percentage;

public:

Student(char n, float per) {

~~name~~ name = n; ~~per = percentage;~~ percentage = per; }

void disp() {

cout << "Name: " << name;

cout << "Percentage: " << percentage; }

~Student() {

cout << "Destroyed." ; }

};

int main() {

Student s1("Omkar", 90);

s1.disp();

return 0;

}

Output:

Name: Omkar

Percentage: 90

Destroyed.

- Copy Constructor

```
#include <iostream>
```

```
using namespace std;
```

```
class Student {
```

```
    char name[20];
```

```
    float percentage;
```

```
public:
```

```
    Student() {
```

```
        name = "Omkar";
```

```
        percentage = 90; }
```

```
    Student(Student &s) {
```

```
        s = s.name;
```

```
        s = s.percentage; }
```

```
    void disp() {
```

```
        cout << "Name: " << name;
```

```
        cout << "Percentage: " << percentage; }
```

```
    ~Student() {
```

```
        cout << "Destroyed: "; }
```

```
    }
```

```
int main() {
```

```
    Student s1;
```

```
    Student s2(s1);
```

```
    s1.disp();
```

```
    s2.disp();
```

```
    return 0;
```

```
}
```

Output: Name: Omkar

Percentage: 90

Name: Omkar

Percentage: 90

Destroyed.

Destroyed.

- ⑧ Define a class 'College' members variable as roll^{no}~~name~~, name, course. MAP using constructor with default value as "Computer Engineering" for course. Accept this data for two objects of class and display the data.

Ans:

```
#include <iostream>
using namespace std;
class College {
    int roll_no;
    string name;
    string course;
public:
    College(int r, string n, string c = "Computer Engineering") {
        roll_no = r;
        name = n;
        course = c;
    }
    void disp() {
        cout << "Roll no: " << roll_no;
        cout << "Name: " << name;
        cout << "Course: " << course;
    }
};

int main() {
    College c1(1, "Omkar");
    College c2(2, "Jael");
    c1.disp();
    c2.disp();
    return 0;
}
```

Output: Roll no: 1
 Name: Omkar
 Course: Computer Engineering.
~~Destroyed~~ Roll no: 2
 Name: Joel

Course: Computer Engineering

Destroyed.

Destroyed.

(4) Write a program to demonstrate constructor overloading.

Ans:

```
#include <iostream>
using namespace std;
class Rectangle {
    int l, w;
public:
    Rectangle() {
        l = 2;
        w = 3;
    }
    Rectangle (int a) {
        l = w = a;
    }
    Rectangle (int a, int b) {
        l = a;
        w = b;
    }
    void disp() {
A = l * b;
        cout << "Area = " << A; A = l * w;
    }
    ~Rectangle() {
int cout << "Destroyed.";
    }
};

int main() {
    Rectangle r1();
    Rectangle r2(7);
    Rectangle r3(4, 5);
    r1.disp();
    r2.disp();
```



```
r3 disp();
```

```
return 0;
```

```
}
```

Output: Area = 6

Area = 49

Area = 20

Destroyed.

Destroyed.

Destroyed.

~~11/9~~

Practical - 6.

Q1 Create a base class ~~de~~ called Person with attributes name and age. Derive a class Student from Person that adds an attribute rollnumber. Write functions to display all details of the student.

Ans:

```
#include <iostream>
using namespace std;
class Person {
protected:
    String name;
    int age;
public:
    Person() {
        name = "Omkar";
        age = 17;
    }
};

class Student : public Person {
private:
    int rollno;
public:
    Student() {
        rollno = 28;
    }
    void disp() {
        cout << "Name is " << name << endl;
        cout << "Age is " << age << endl;
        cout << "Roll no is " << rollno << endl;
    }
};

int main() {
    Student s1;
    s1.disp();
    return 0;
}
```


Q.2. Create two base classes Academic and Sports.

- ★ - Academic class contains marks of a student.
- Sports class contains ~~marks~~ sports score.
- Create a derived class Result that inherits from both Academic and Sports. Write a function to calculate the total score and display details.

Ans:

```
#include <iostream>
using namespace std;
class Academic {
protected:
float m1, m2;
Academic() {
m1 = 26;
m2 = 25;
}
};
class Sports {
protected:
int score;
Sports() {
sportsscore = 25;
}
};
class Result : public Academic, public Sports {
float r;
public:
void total() {
r = (m1 + m2) / 2;
}
void disp() {
cout << "Result: " << r << endl;
}
```

```

    cout << "Sports score: " << score;
}
};

int main() {
    Result R1;
    R1.total();
    R1.disp();
    return 0;
}

```

Q.3. Create a class vehicle with attributes like brand and model. Derive ~~two~~ a class car from vehicle which adds an attribute type (e.g., sedan, SUV). Further derive a class ElectricCar from car which adds battery capacity. Write functions to display all the details.

Ans:

```

#include <iostream>
using namespace std;
class vehicle {
protected:
    string brand;
    string model;
    vehicle() {
        brand = "BMW";
        model = "m3";
    }
};

class Car : public vehicle {
protected:
    string attribute;
    Car() {
        attribute = "Sedan";
    }
};

```



```

class ElectricCar : public Car {
private:
    string battery;
public:
    ElectricCar() {
        battery = "66kWh";
    }
    void disp() {
        cout << "Brand: " << brand << endl;
        cout << "Model: " << model << endl;
        cout << "Attribute: " << attribute << endl;
        cout << "Battery: " << battery;
    }
};

int main() {
    ElectricCar E1;
    E1.disp();
    return 0;
}

```

- Q.4. Create a base class Employee with attributes empid and name. Derive two classes Manager and Developer from Employee. Manager has an attribute department and Developer has an attribute programming language. Write functions to display details for both.

Ans:

```

#include <iostream>
using namespace std;
class Employee {
protected:
    int empid;
    string name;
public:

```

```
Employee() {  
    empid = 28;  
    name = "Omkar";  
    void disp() {  
        cout << "Employee ID: " << empid << endl;  
        cout << "Name: " << name << endl;  
    }  
};
```

```
class Manager : public Employee {  
    string dept;  
    public:  
    Manager(string d) {  
        dept = d;  
    }  
    void md() {  
        disp();  
        cout << "Department  
Programming Language: " << dept << endl;  
    }  
};
```

```
class Developer : public Employee {  
    string proleng;  
    public:  
    Developer(string p) {  
        proleng = p;  
    }  
    void get() {  
        disp();  
        cout << "Programming language: " << proleng;  
    }  
};
```

```
int main() {  
    Manager m1("IT");  
    m1.md();  
}
```



```
Developer d1 ("c++");
d1.get();
return 0;
}
```

- Q.5. Combine multilevel and multiple Inheritance. Create a base class Person with attributes name and age. Derive class Student from Person. Create two classes Sports and Academics. Derive class Result from Student and Sports. Write functions to calculate and display total marks and sports score with student details.

Ans:

```
#include <iostream>
using namespace std;
class Person {
protected:
    string name;
    int age;
public:
    Person() {
        name = "Omkar";
        age = 17;
    }
};

class Student : virtual public Person {
protected:
    int rollno;
public:
    Student(int r) {
        rollno = r;
    }
};

class Academic : virtual public Person {
protected:
```

```

int marks;
public:
Academic(int c) {
    marks = c;
}
};

class sports : virtual public Person {
protected:
int score;
public:
sports(int s) {
    score = s;
}
};

class Result : public Student, public Academic, public sports {
int total;
public:
Result(int r, int c, int s) : Student(r), Academic(c), sports(s) {
    total = marks + score;
}

void disp() {
    cout << "Name : " << name << endl;
    cout << "Age : " << age << endl;
    cout << "Roll no: " << rollno << endl;
    cout << "Total Marks: " << total;
}
};

int main() {
    Result r1(28, 82, 90);
    r1.disp();
    return 0;
}

```


26/9/25

26-9-25

classmate

Date

Page

- Practical - 7

Q. Write a program to demonstrate the compile time polymorphism (Function overloading and operator overloading - Unary).

① Write a program using function overloading to calculate the area of a laboratory (which is rectangular in shape) and area of ~~at~~ classroom (which is square in shape).

Ans:

```
#include <iostream>
```

```
using namespace std;
```

```
class Area {
```

```
public:
```

```
void area (int a, int b) {
```

```
    cout << "Area of laboratory : " << a*b << endl;
```

```
void area (int a) {
```

```
    cout << "Area of classroom : " << a*a;
```

```
};
```

```
int main () {
```

```
    Area a1;
```

```
    a1.area(12,6);
```

```
    a1.area(6);
```

```
    return 0;
```

```
}
```

- ② Write a program using function overloading to calculate the sum of 5 float values and sum of 10 integer values

Ans:

```
#include <iostream>
using namespace std;
class Cal {
public:
    void sum (float a[5]) {
        int i;
        float s=0;
        for(i=0; i<5; i++){
            s+= a[i];
        }
        cout<<" Sum: " << s << endl;
    }
    void sum (int a[10]) {
        int i, s=0;
        for (i=0; i<10; i++){
            s+= a[i];
        }
        cout << "Summ: " << s << endl;
    }
};

int main () {
    Cal c1;
    int a[] = {1, 5, 2, 6, 8, 6, 5, 22, 8, 543};
    float b[] = {2.3, 5.7, 8.9, 5.5, 4.13};
    c1.sum (b);
    c1.sum (a);
    return 0;
}
```


- ③ Write a program to implement Unary - operator when used with the object so that the numeric data member of the class is negated.

Ans:

```
#include <iostream>
using namespace std;
class Number {
    int a;
public:
    void accept() {
        cout << "Enter value of a: ";
        cin >> a;
    }
    void operator-() {
        a = -a;
    }
    void disp() {
        cout << "value: " << a;
    }
};

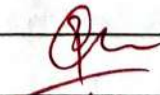
int main() {
    Number n1;
    n1.accept();
    -n1;
    n1.disp();
    return 0;
}
```

- ④ Write a program to implement the unary ++ operator (for pre increment and post increment) when used with the object so that the numeric data member of the class is incremented.

Ans:

```
#include <iostream>
using namespace std;
class Number {
    int a;
    public:
        void accept() {
            cout << "Enter a: ";
            cin >> a;
        }
        void operator ++() {
            a = ++a;
        }
        void disp() {
            cout << "Value: " << a;
        }
};

int main() {
    Number n1;
    n1.accept();
    ++n1;
    n1.disp();
    return 0;
}
```


10/11

- Practical - 8.

Q. Write a program to demonstrate the compile time and run time polymorphism (operator overloading Binary).

① Write a program to overload the '+' operator so that two strings can be concatenated. Eg: "xyz" + "pqr" then output will "xyzpqr".

Ans:

```
#include <iostream>
using namespace std;
class join {
    string a, b;
public:
    void accept() {
        cout << "Enter first string: ";
        cin >> a;
        cout << "Enter second string: ";
        cin >> b;
    }

    string concatenated() {
        return a + b;
    }
};

int main() {
    join j;
    j.accept();
    string result = j.concatenated();
    cout << "Concatenated string is: " << result << endl;
    return 0;
}
```

- Q) Write a program to create a base class ILogin having data members name and password. Declare accept() function virtual. Derive EmailLogin and MembershipLogin classes from ILogin. Display Email Login and membership login details of the employee.

Ans:

```
#include <iostream>
using namespace std;
class ILogin {
protected:
    string name, pass;
public:
    virtual void accept() {}
    virtual void display() {}
};

class EmailLogin : public ILogin {
public:
    void accept() {
        cout << "Enter Email and Pass: ";
        cin >> name >> pass;
    }
    void display() {
        cout << "Email: " << name << " /n Pass: " << pass << endl;
    }
};

class MembershipLogin : public ILogin {
public:
    void accept() {
        cout << "Enter Member Name and Pass: ";
        cin >> name >> pass;
    }
    void display() {
        cout << "Member: " << name << " Pass: " << pass << endl;
    }
};
```



```
int main () {  
    ILogin *p;  
    EmailLogin e;  
    Membership login m;  
    p = &e;  
    p->accept();  
    p->display();  
    p = &m;  
    p->accept();  
    p->display();  
}
```


10/11

Experiment - 9

Q. Write a program to perform various operations on file.

Q. Write a program to copy the contents of one file into another. Open "First.txt" in read (ios::in) mode and "Second.txt" file in write (ios::out) mode. Copy the contents of "First.txt" into "Second.txt". Assume "First.txt" is already created.

Ans:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
int main() {
    char ch;
    ifstream fin("first.txt");
    ofstream fout("destination.txt");
    if (!fin) {
        cout << "ERROR in opening source file " << endl;
        return 1;
    }
    if (!fout) {
        cout << "ERROR in opening destination file " << endl;
        fin.close();
        return 1;
    }
    while (fin.get(ch)) {
        fout << ch;
    }
    fin.close();
    fout.close();
    cout << "File is successfully copied";
    return 0;
}
```


⑥ Write a C++ program to count words using File Handling.

Ans:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
int main() {
    string word;
    int count = 0;
    ifstream fin("source.txt");
    ofstream fout("destination.txt");
    if (!fin) {
        cout << "Error in opening source file." << endl;
        return 1;
    }
    if (!fout) {
        cout << "Error in opening destination file." << endl;
        fin.close();
        return 1;
    }
    while (fin >> word) {
        count++;
        fout << word;
    }
    fin.close();
    fout.close();
    cout << "Total no. of words = " << count;
    return 0;
}
```

Q Write a C++ program to count occurrence of a given word using file handling.

```
Ans: #include <iostream>
#include <fstream>
#include <string>
using namespace std;
int main () {
    string word;
    string a;
    int count = 0;
    int a_count = 0;
    ifstream fin("source.txt");
    ofstream fout("destination.txt");
    cout << "Enter a word: ";
    cin >> a;
    if (!fin) {
        cout << "Error in opening source file" << endl;
        return 1;
    }
    if (!fout) {
        cout << "Error in opening destination file" << endl;
        fin.close ();
        return 1;
    }
    while (fin >> word) {
        fout << word;
        cout << " ";
        if (word == a) {
            a_count ++;
        }
    }
}
```



```

    fin.close();
    fout.close();
    cout<<"Total no. of repeats = "<<a_count;
    return 0;
}

```

Q Write a C++ program to count digits and spaces using file handling.

Ans:

```

#include <iostream>
#include <fstream>
#include <string>
using namespace std;
int main()
{
    char ch;
    int digitcount=0;
    int spacecount=0;
    ifstream fin("source.txt");
    ofstream fout("destination.txt");
    if(!fin)
    {
        cout<<"Error in opening destination file"<<endl;
        fin.close();
        return 1;
    }
    while(fin.get(ch))
    {
        fout<<ch;
        if(isdigit(ch))
            digitcount++;
    }
    if(ispace(ch))
        spacecount++;
}
}

```

```
fin.close();
```

```
fout.close();
```

```
cout << "File is successfully copied." << endl;
```

```
cout << "Number of digits: " << digitcount << endl;
```

```
cout << "Number of spaces: " << spacecount << endl;
```

```
return 0;
```

```
}
```

Qm
10/11

- Exp-10.
- Write a program to implement a function template and class template.

① Write a C++ program to find sum of array elements using function template (eg. Pass Integer, Float and Double array of 10 elements).

Ans:

```
#include <iostream>
using namespace std;
template <class t> t func (t a[], int n) {
    t sum = 0;
    int i;
    for (i = 0; i < n; i++) {
        sum = sum + a[i];
    }
    return sum;
}
int main () {
    int a1[3] = {2, 0, 2};
    float a2[3] = {1.2, 1.2, 2.4};
    double a3[3] = {10.5, 20.5, 30.5};
    cout << "Sum of integer array is: "
         << func(a1, 3) << endl;
    cout << "Sum of float array is: " << func(a2, 3) << endl;
    cout << "Sum of double array is: " << func(a3, 3) << endl;
}
```

Output:

Sum of integer array: 4

Sum of float array: 4.8

Sum of double array: 61.5

2) Write a C++ program of square function using template specialization.

nd:

```
#include <iostream>
```

```
using namespace std;
```

```
template <class T> T square (T x) {
```

```
    T sum = 0
```

```
    sum = x * x;
```

```
    return sum; }
```

```
template <> string square <string>
```

```
    (string ss) {
```

```
    return (ss+ss);
```

```
}
```

```
int main () {
```

```
    int i = 2, ii;
```

```
    string ww("MIT");
```

```
    ii = square <int>(i);
```

```
    cout << "Square of i is: " << ii << endl;
```

```
    cout << "Square of given string is: " << square << string(ww) << endl;
```

Output :

Square of 2 is: 4

Square of given string is : MITMIT.

③ Write a C++ program to build simple calculator using class template.

Ans:

```
#include <iostream>
#include <math>
using namespace std;

template <class T>
class calculator {
private:
    T num1, num2;
public:
    calculator (Tn1, Tn2) {
        num1 = n1;
        num2 = n2;
    }

    T add () {
        return num1 + num2;
    }

    T sub () {
        return num1 - num2;
    }

    T mul () {
        return num1 * num2;
    }

    T divide () {
        return num1 / num2;
    }

    T mod () {
        return num1 % num2;
    }

    void disp () {
        cout << "Numbers: " << num1 << " and " << num2 << endl;
    }
};

int main () {
    double a, b;
    int ch;
```

```

cout << "Enter two numbers: ";
cin >> a >> b;
calculator <double> calc(a, b);
cout << "Addition: " << calc.add() << endl;
cout << "Subtraction: " << calc.sub() << endl;
cout << "Multiplication: " << calc.mul() << endl;
cout << "Division: " << calc.divide() << endl;
cout << "Modulo: " << calc.mod() << endl;
return 0;

```

- ④ Write a C program to implement push and pop methods from stack using class template.

Ans:

```

#include <iostream>
using namespace std;
template <class T>
class Stack{
    T arr[100];
    int top;
public:
    Stack() { top = -1; }
    void push(T val){
        if (top < 99){
            arr[++top] = val;
            cout << "Pushed: " << val << endl;
        }
        else {
            cout << "Stack overflow!!" << endl;
        }
    }
    void pop(){
        if (top >= 0){
            cout << "Popped: " << arr[top] << endl;
        }
        else {
            cout << "Stack overflow" << endl;
        }
    }
    void display(){
        cout << "Stack: ";

```



```
for (int i=0; i<=top; ++i)
    cout << arr[i] << " ";
    cout << endl;
}
```

```
};
int main () {
    Stack <int> s;
    s.push(10);
    s.push(20);
    s.push(30);
    s.display();
    s.pop();
    s.pop();
    s.pop(); s.pop();
    return 0;
}
```

Output: Pushed: 10
Pushed: 20
Pushed: 30
Stack: 10 20 30
Popped: 30
Popped: 20
Popped: 10
Stack underflow.


10/11

Practical - 11.

- Q. Write a C++ program to implement generic vectors with functions to create the vector, modify an element, multiply by a scalar, and display the vector.

Ans:

```
#include <iostream>
#include <vector>
using namespace std;

template <typename T>
class GenericVector {
    vector<T> data;
public:
    void create (int n) {
        data.resize (n);
        cout << "Enter " << n << " elements: " << endl;
        for (int i = 0; i < n; i++) {
            cin >> data[i];
        }
    }
    void modify (int index, T value) {
        if (index >= 0 && index < data.size()) {
            data[index] = value;
        }
    }
    void multiply (T scalar) {
        for (auto &el : data)
            el *= scalar;
    }
    void display () {
        cout << "Vector: ";
        for (const auto &el : data)
            cout << el << " ";
        cout << endl;
    }
};
```



```
int main() {
    GenericVector<int> vec;
    int n;
    cout << "Enter no. of elements: ";
    cin >> n;
    vec.create(n);
    vec.display();
    cout << "Modify element at index 1 to 20: " << endl;
    vec.modify(1, 20);
    vec.display();
    cout << "Multiply all elements by 2: " << endl;
    vec.multiply(2);
    vec.display();
    return 0;
}
```

3

Output:

Enter number of elements: 3

Enter 3 elements:

10 15 30

vector: 10 15 30

Modify element at index 1 to 20:

vector: 10 20 30

Multiply all elements by 2:

Vector: 20 40 60.



- Exp-11

- Without Iterator.

Ans.

```
#include <iostream>
```

```
#include <vector>
```

```
#include <ctype>
```

```
using namespace std;
```

```
int main() {
```

```
    vector<char> v10;
```

```
    unsigned int i;
```

```
    cout << "size = " << v10.size() << endl;
```

```
    for (i = 0; i < v10.size(); i++) {
```

```
        for (i = 0; i < v10.size(); i++) {
```

```
            cout << v10[i] << " ";
```

```
            cout << "\n\n";
```

```
            cout << "Expand vectors: \n";
```

```
            for (i = 0; i < v10.size(); i++) {
```

```
                v10.push_back (i + 40 + 'a');
```

```
            cout << "Size Now = " << v10.size() << endl;
```

```
            cout << "Current contents: \n";
```

```
            for (i = 0; i < v10.size(); i++) {
```

```
                cout << v10[i] << " ";
```

```
                cout << "\n\n";
```

```
            for (i = 0; i < v10.size(); i++) {
```

```
                v10[i] = to toupper (v10[i]);
```

```
            cout << "Modify contents: \n";
```

```
            for (i = 0; i < v10.size(); i++) {
```

```
                cout << v10[i] << " ";
```

```
            cout << "\n\n";
```

```
        return 0;
```


Output:

Initial size = 5

Original contents: a b c d e

expand vectors...

size now = 10

current contents: a b c d e f g h i j

Modifying contents to uppercase.

Modified contents: A B C D E F G H I J.

Qm
10/11

- Practical - 12

② STL Operations : Stack, Queue, Sorting, Searching.

- Write C++ program using STL.

① Implement Stack.

Ans:

```
#include <iostream>
```

```
#include <stack>
```

```
using namespace std;
```

```
int main(){
```

```
    stack<int> s;
```

```
    s.push(10);
```

```
    s.push(20);
```

```
    s.push(30);
```

```
    cout << "Stack top: " << s.top() << endl;
```

```
    s.pop();
```

```
    cout << "Stack top after pop: " << s.top() << endl;
```

```
    return 0;
```

Output:

Stack top: 30

Stack top after pop: 20

(b) Implement Queue.

Ans:

```
#include <iostream>
#include <queue>
using namespace std;
int main(){
    q.push(100);
    q.push(200);
    q.push(300);
    cout<<"Queue front: "<<q.front()<<endl;
    q.pop();
    cout<<"Queue front after pop: "<<q.front()<<endl;
    return 0;
}
```

Output:

Queue front: 100

Queue front after pop: 200.


10/11